

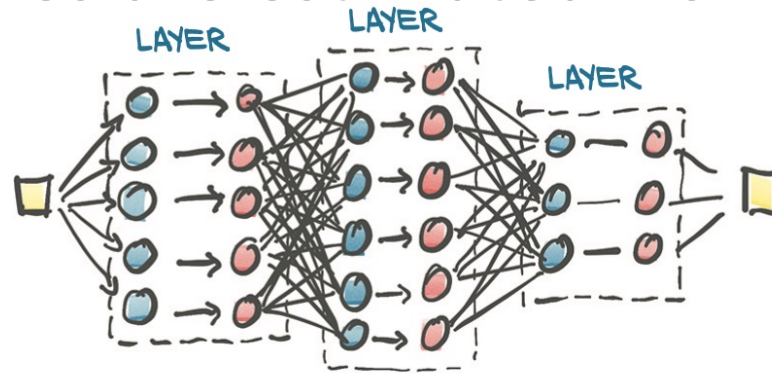
Lecture 5: Using a Neural Network to Fit the Data (1)

Overview

- Understand how algorithms can learn from data
- Reframe learning as parameter estimation
- Generalization and overfitting

Understand Learning

- **Learn** from data = **fit** the data (by using an algorithm)
 - This process always involves a function with a number of unknown parameters whose values are estimated from data: in short, a *model*.

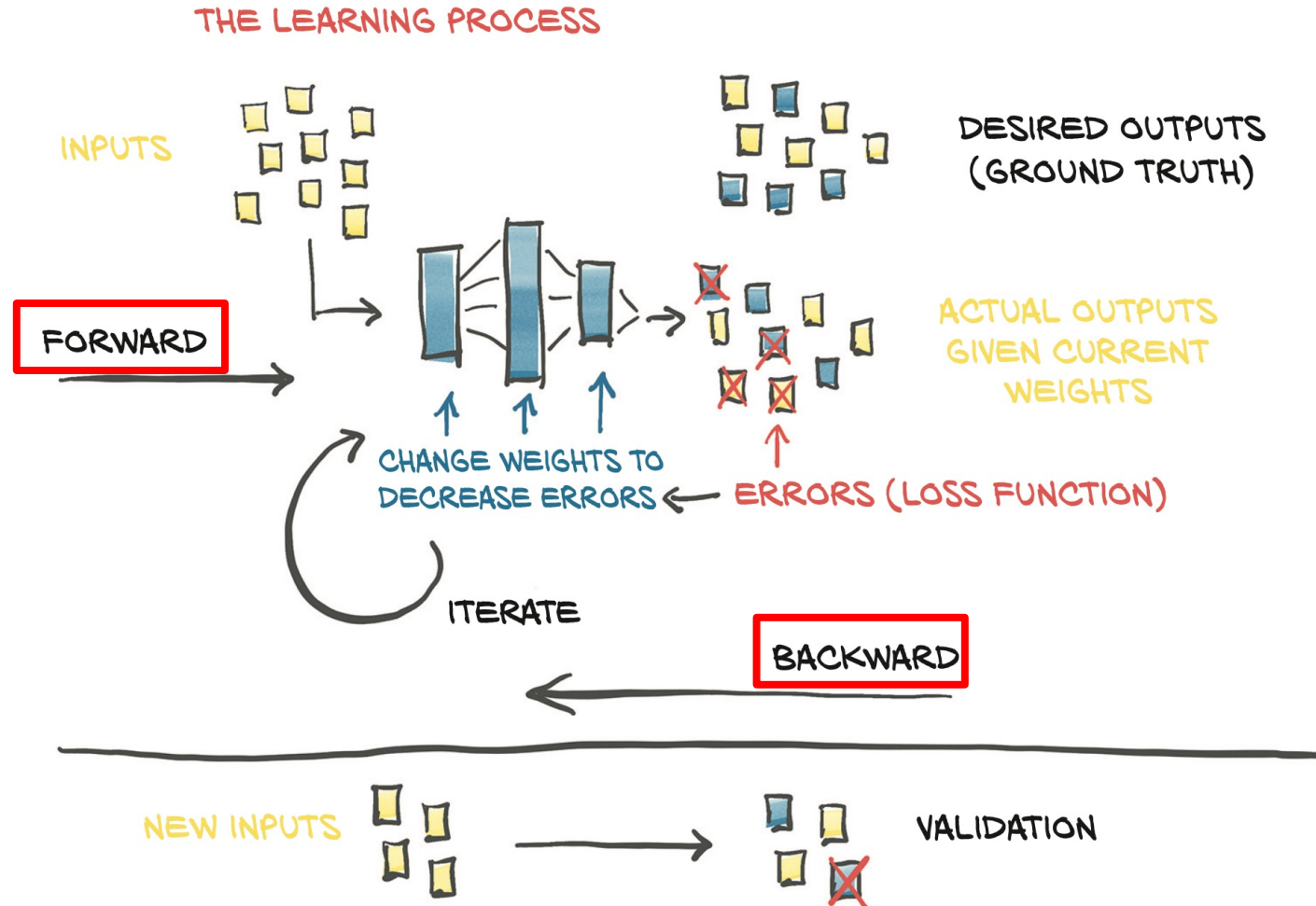


A NEURAL NETWORK

$$\text{OUTPUT} \uparrow O = \tanh \left(w_n \left(\dots \tanh \left(w_2 \left(\tanh \left(w_1 x + b_1 \right) + b_2 \right) \right) \dots + b_n \right) \right)$$

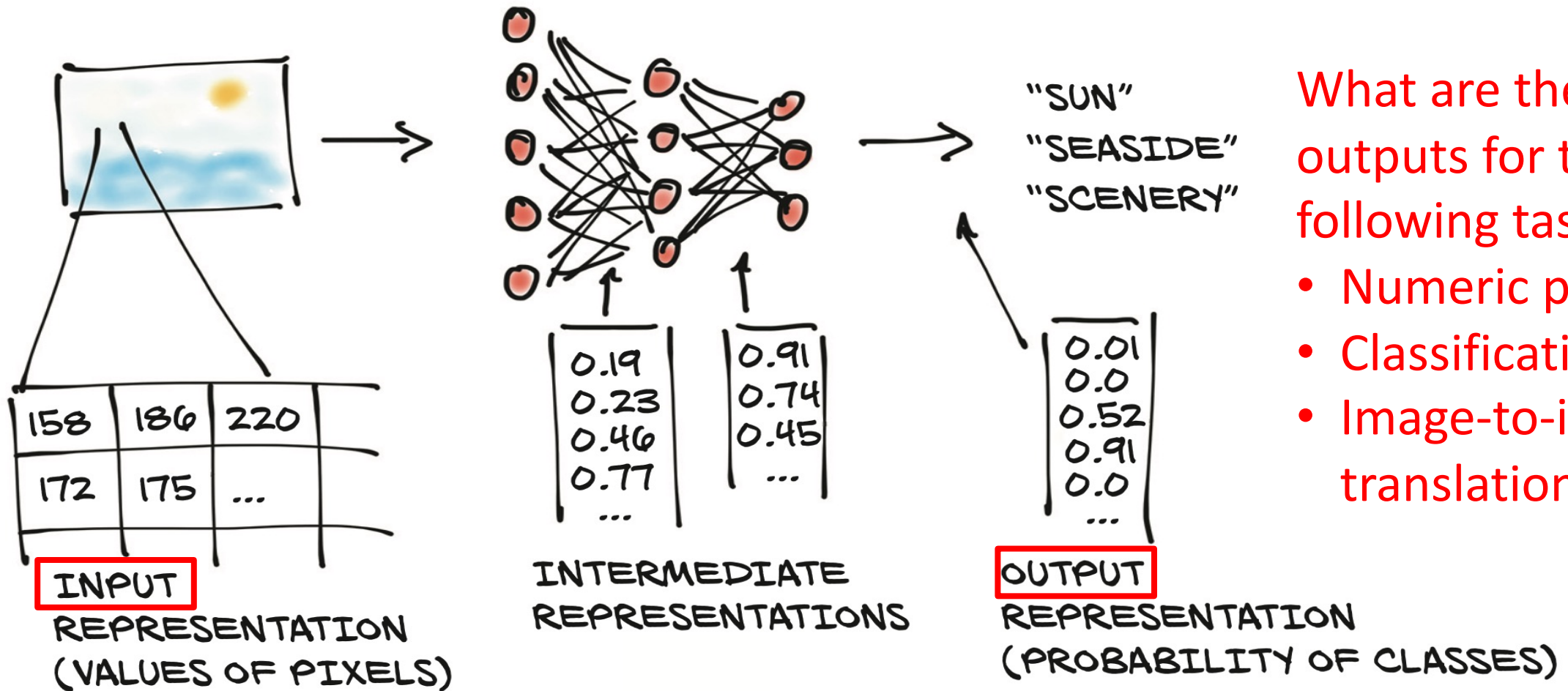
Diagram illustrating the mathematical representation of a neural network layer. The equation shows the output O (labeled OUTPUT with an upward arrow) calculated using the $\tanh$ activation function. The input x (labeled INPUT with a downward arrow) is combined with learned parameters $w_1, w_2, \dots, w_n$ and biases $b_1, b_2, \dots, b_n$ (labeled LEARNED PARAMETERS with upward arrows) to produce the output.

The Learning Process



- Learning is just parameter estimation (learning from errors)
 - Forward pass
 - Backward pass

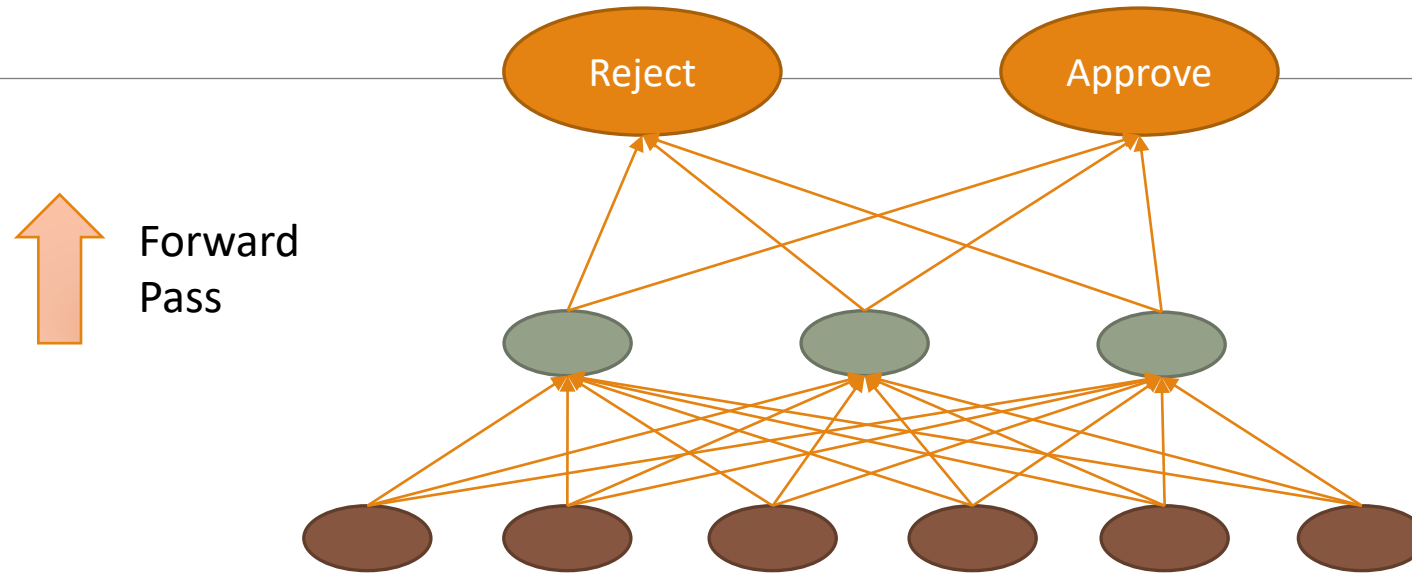
The Learning Process



What are the model outputs for the following tasks?

- Numeric prediction
- Classification
- Image-to-image translation

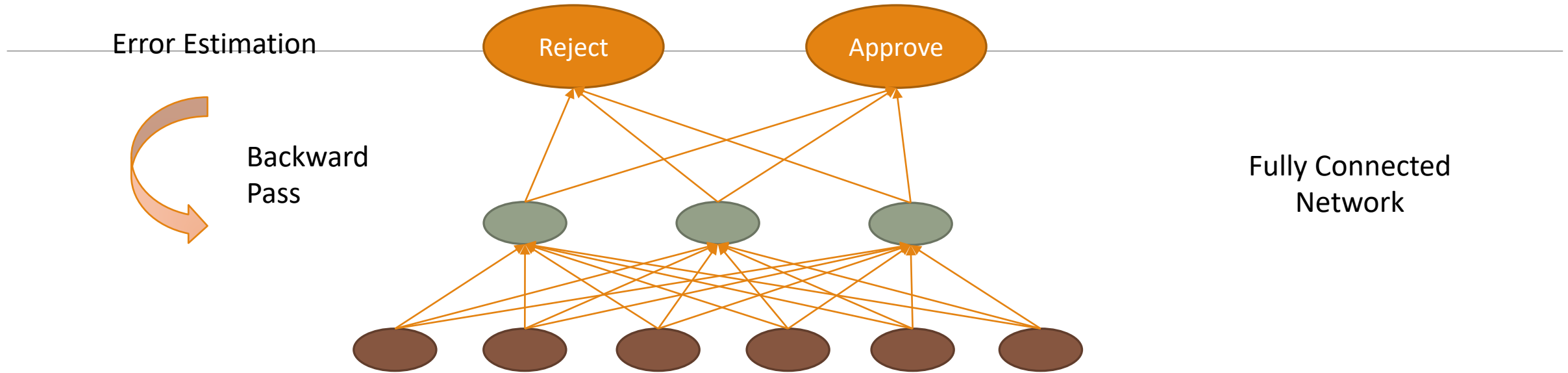
Neural Network: **Forward Pass**



- Step 1: Model initialization: Randomly set initialized neural network
- Step 2: Feed forward: The model is fed input data

CID	Credit Score	Income (10,000)	Term	Age	Gender	Number of Children	Status
1	6	2	1	35	0	0	Reject
2	7.5	5	3	38	0	2	Approve
3	4	3.5	2	23	1	4	Reject
...							
100000	8	2.5	1	35	0	1	Approve

Neural Network: Backward Pass



- Step 3: Calculate the error: How far is your model output from the actual value.
- Step 4: Compute gradient of the error: apply the chain rule to compute gradient with respect to the parameters
- Step 5: Update the parameters: Update the parameters (weights and biases) until the error becomes minimum.

The Learning Process

- **Forward pass**

- Given input data and initial values for the weights, the model is fed input data

- **Backward pass**

- A measure of the error is evaluated by comparing the outputs to the ground truth
 - The gradient of the error with respect to the parameters is computed using the chain rule
 - Optimize the parameter of the model: the value of the weights is then updated in the direction that leads to a decrease in the error.
- The procedure is repeated until the error falls below an acceptable level.

A Thermometer Example

- We brought back a fancy, wall-mounted analog thermometer. It looks great, and it's a perfect fit for our living room. Its only flaw is that it doesn't show units.
- Not to worry, we've got a plan: we'll build a dataset of readings and corresponding temperature values in Celsius

```
t_c = [0.5, 14.0, 15.0, 28.0, 11.0, 8.0, 3.0, -4.0, 6.0, 13.0, 21.0]
t_u = [35.7, 55.9, 58.2, 81.9, 56.3, 48.9, 33.9, 21.8, 48.4, 60.4, 68.4]
t_c = torch.tensor(t_c)
t_u = torch.tensor(t_u)
```

A Thermometer Example

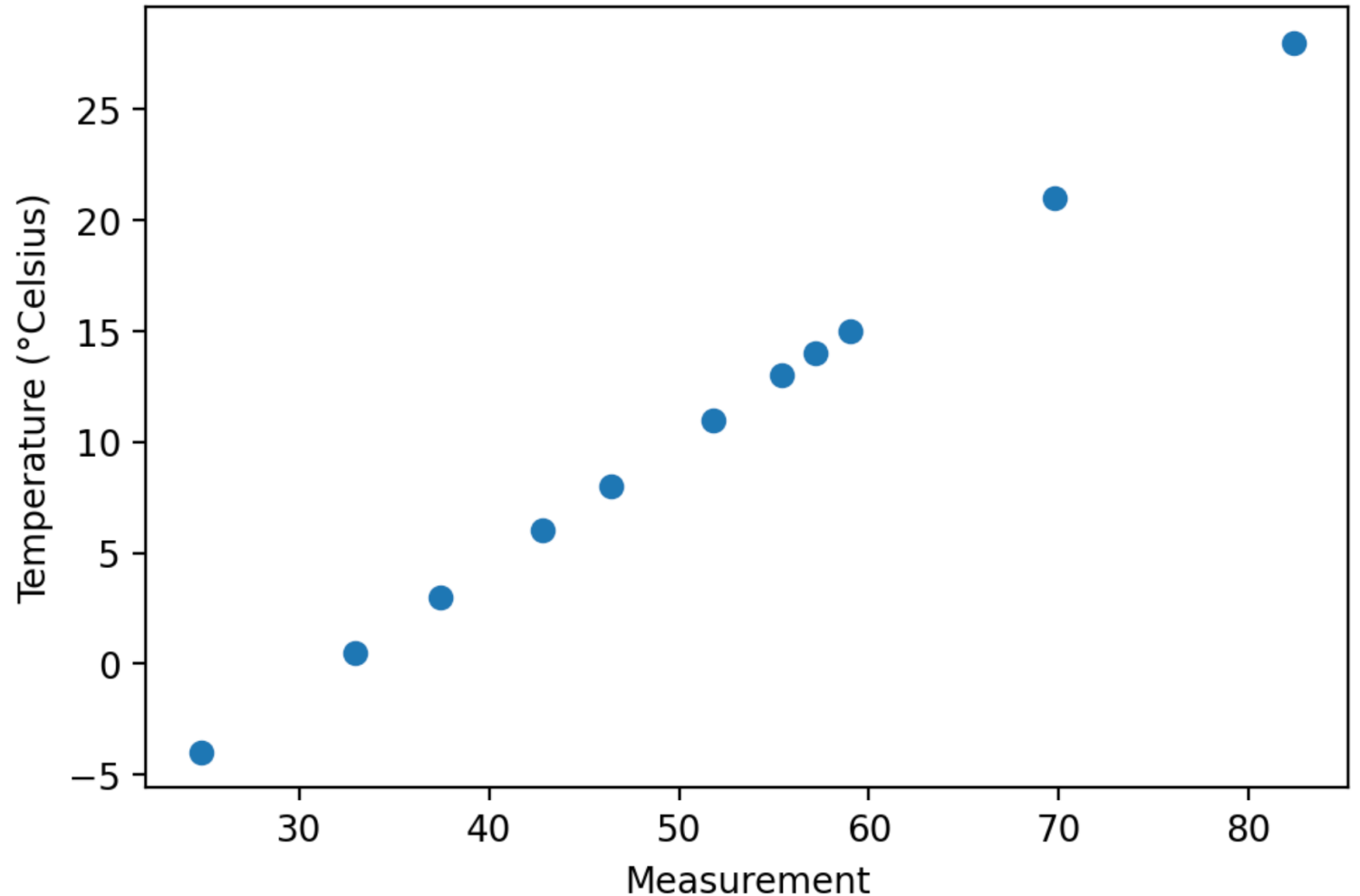
```
t_c = [0.5, 14.0, 15.0, 28.0, 11.0, 8.0, 3.0, -4.0, 6.0, 13.0, 21.0]
t_u = [35.7, 55.9, 58.2, 81.9, 56.3, 48.9, 33.9, 21.8, 48.4, 60.4, 68.4]
t_c = torch.tensor(t_c)
t_u = torch.tensor(t_u)
```

- The `t_c` values are temperatures in Celsius
- The `t_u` values are read from our new thermometer with unknown units

A Thermometer Example

- Visualizing the data

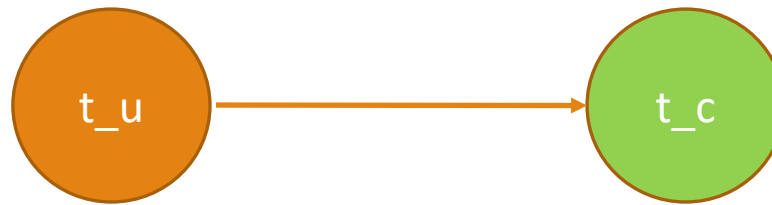
Do you identify any patterns?



A Thermometer Example

- Choosing a linear model as a first try

$$\text{output } t_c = w * \text{input } t_u + b$$



- Two parameters in this linear model
 - w : weight
 - b : bias

A Thermometer Example

- **Forward pass**

- Given input data and initial values for the weights, the model is fed input data
- Generate **predicted t_c**

A Thermometer Example

■ Backward pass

1. A measure of the error is evaluated by comparing the outputs to the ground truth
2. The gradient of the error with respect to the parameters is computed using the chain rule
3. Optimize the parameter of the model: the value of the weights is then updated in the direction that leads to a decrease in the error.

A Thermometer Example

Backward pass: 1. A measure of the error is evaluated by comparing the outputs to the ground truth

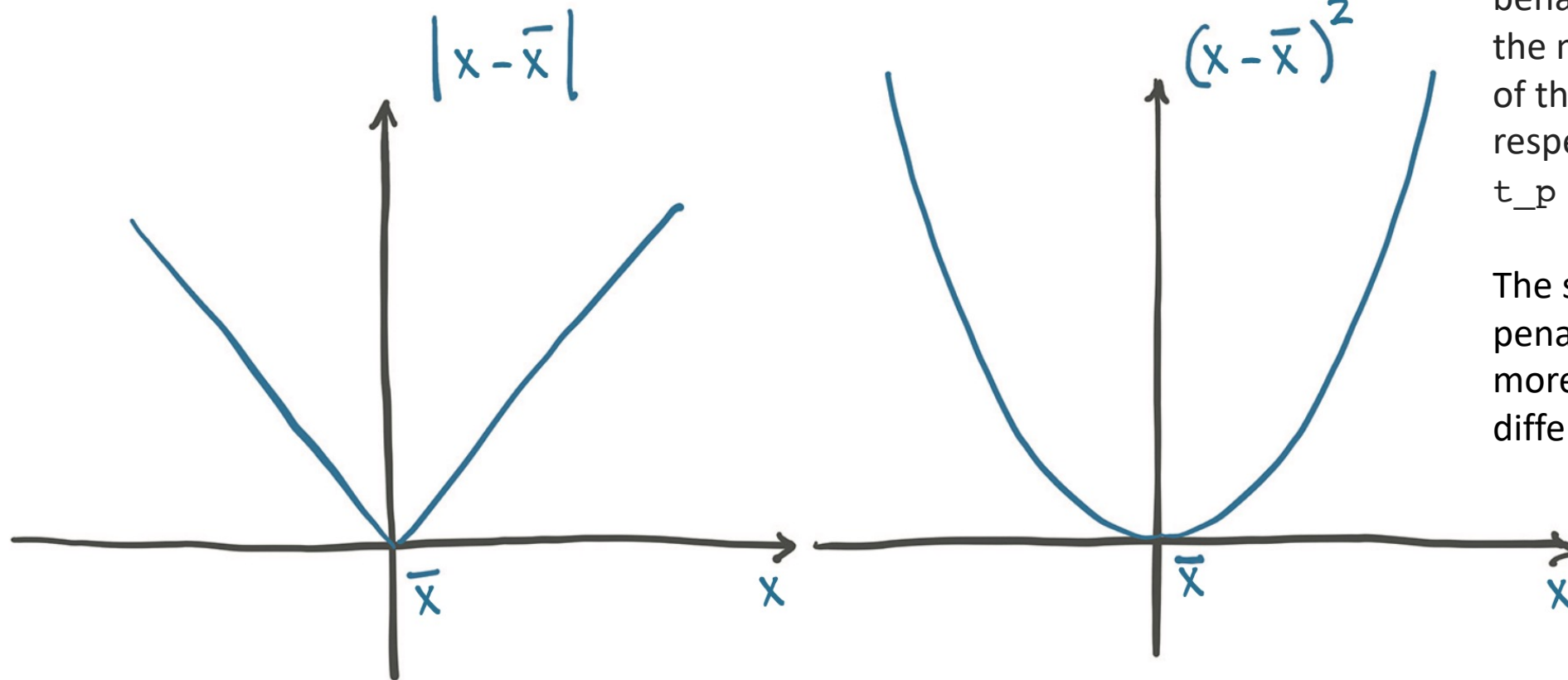
- **Goal—minimize the error:** Our optimization process should therefore aim at finding w and b so that the loss function is at a minimum (the predicted t_c is close to the actual t_c)
 - We need to exactly define a measure of the error
 - *loss function*—should be high if the error is high and should ideally be as low as possible for a perfect match

A Thermometer Example

- A **loss function** (or **cost function**) is a function that computes a single numerical value that the learning process will attempt to minimize
 - The calculation of loss typically involves taking the difference between the model outputs and actual values
 - Absolute error loss: $|t_p - t_c|$
 - Squared error loss: $(t_p - t_c)^2$
- Define the objective of learning

A Thermometer Example

■ Loss function



The square of the differences behaves more nicely around the minimum: the derivative of the error-squared loss with respect to t_p is zero when t_p equals t_c .

The square difference also penalizes wildly wrong results more than the absolute difference does

A Thermometer Example

■ Python Codes:

■ Define the model as a Python function

- $\text{Output} = w * t_u + b$
- t_u , w , and b to be the input tensor, weight parameter, and bias parameter

■ Define the loss function (*mean square loss*)

- $\text{squared_diffs} = (t_p - t_c)^2$
- `return squared_diffs.mean()`

A Thermometer Example

- Initialize the parameters and invoke the model

```
w = torch.ones(())  
b = torch.zeros(())
```

```
t_p = model(t_u, w, b)  
t_p
```

```
tensor([32.9000, 57.2000, 59.0000, 82.4000, 51.8000, 46.4000, 37.4000, 24.8000,  
        42.8000, 55.4000, 69.8000])
```

A Thermometer Example

- Check the value of the loss

```
loss = loss_fn(t_p, t_c)  
loss
```

```
tensor(1681.5272)
```

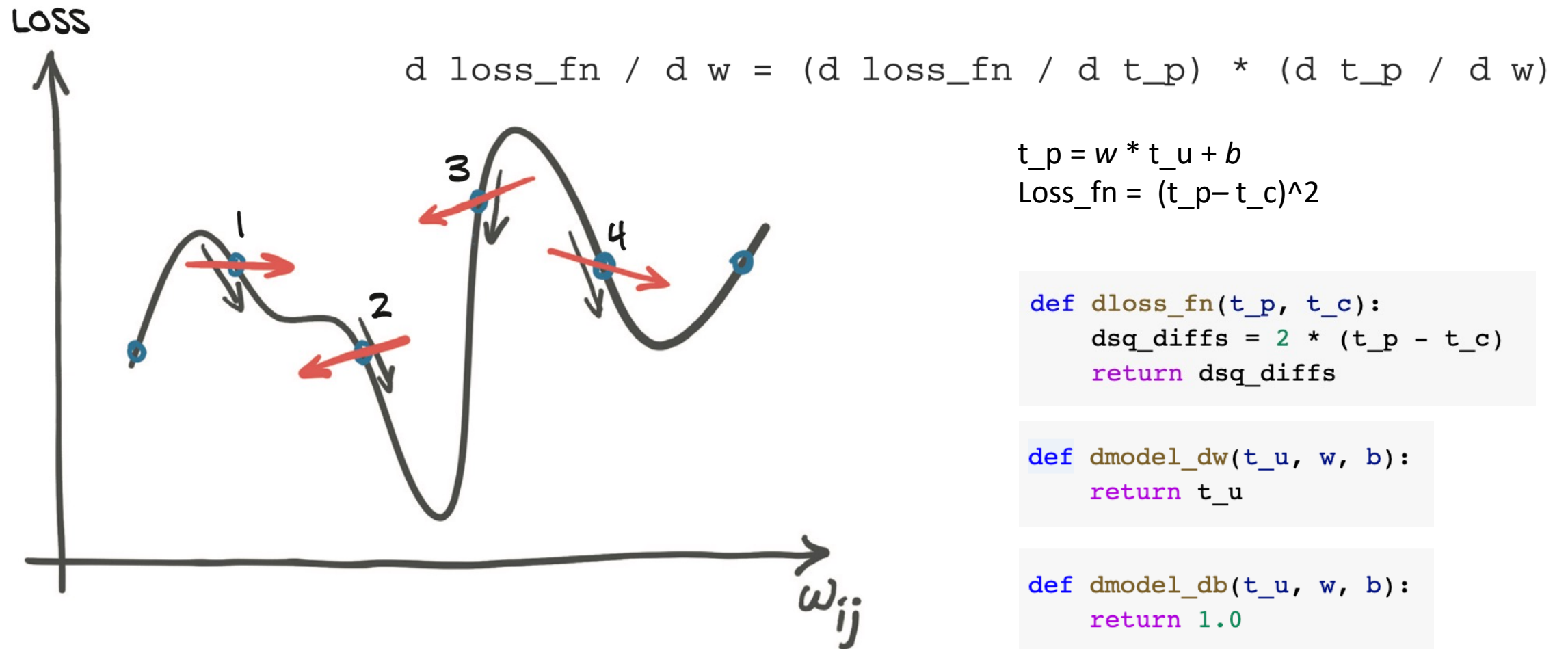
- We implemented the model and the loss. We've finally reached the meat of the example: how do we estimate w and b such that the loss reaches a minimum?

A Thermometer Example

- Optimize the loss function with respect to the parameters
 - Compute the gradient
 - Optimizer
 - *Gradient descent* optimizer
 - *Adam* optimizer
- Suppose we are in front of a machine sporting two knobs, labeled w and b . We are allowed to see the value of the loss on a screen, and we are told to minimize that value
 - Rotate both knobs in their direction of decreasing loss



A Thermometer Example



Gradient: slope of error (moving down along that slope towards some minimum level of error)

A Thermometer Example

Backward pass: 2. The gradient of the error with respect to the parameters is computed using the chain rule

- We compute the individual derivatives of the loss with respect to each parameter

```
# Define the gradient function
```

```
def grad_fn(t_u, t_c, t_p, w, b):
```

```
    dloss_dtp = dloss_fn(t_p, t_c)
```

```
    dloss_dw = dloss_dtp * dmodel_dw(t_u, w, b)
```

```
    dloss_db = dloss_dtp * dmodel_db(t_u, w, b)
```

```
    return torch.stack([dloss_dw.mean(), dloss_db.mean()])
```

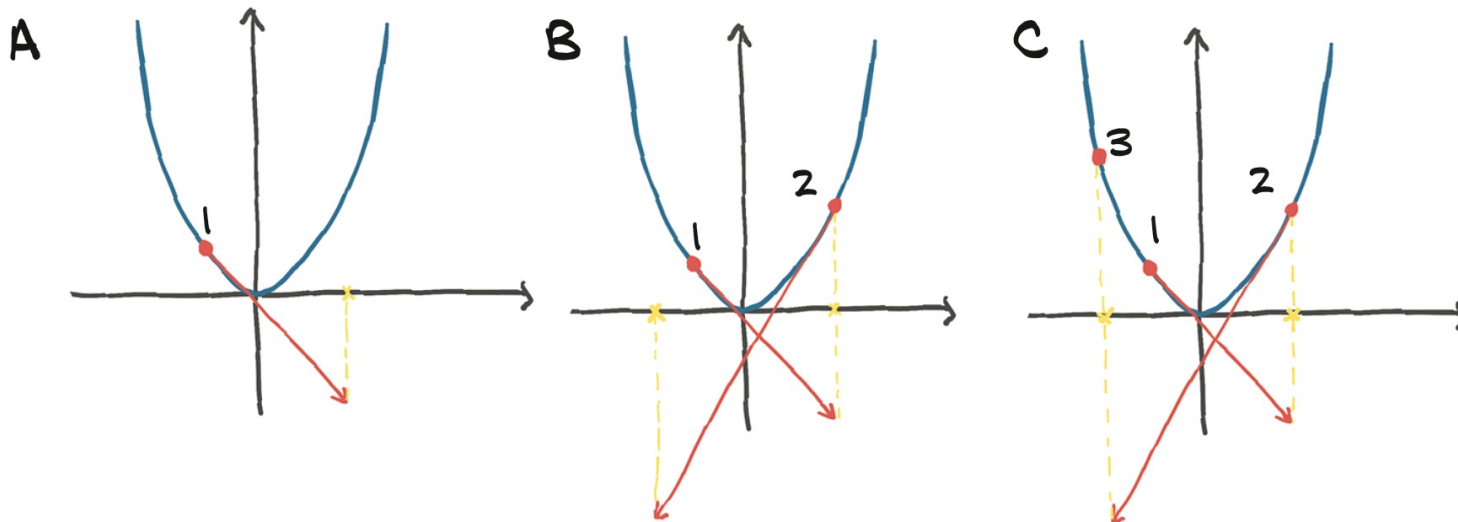
→ The gradient depends on both
→ the loss function and model

A Thermometer Example

- **Backward pass: 3. Optimize the parameter** of the model: the value of the weights is then updated in the direction that leads to a decrease in the error.
- Apply the learning rate: It is also wise to change the parameters slowly in general, because the rate of change could be dramatically different at a distance from the neighborhood of the current w value.
 - We typically should scale the rate of change by a small factor (i.e. 0.001)
- **Learning rate:** The learning rate is a hyperparameter that controls how much to change the model in response to the estimated error each time the model weights are updated

A Thermometer Example

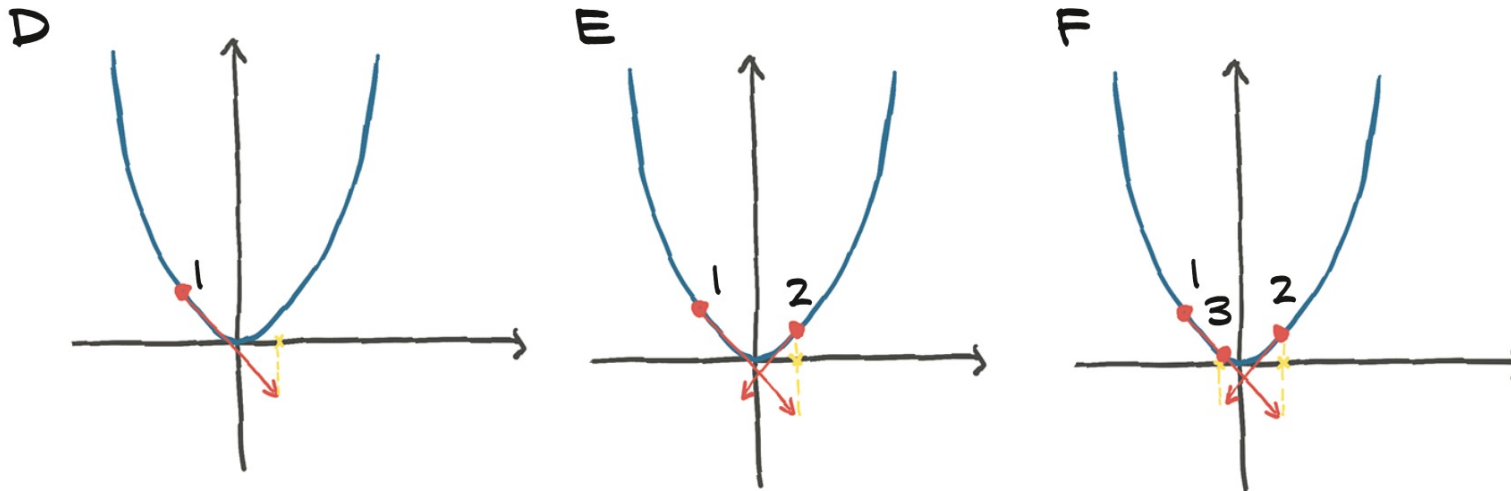
- The effect of learning rate:
 - A big learning rate: the model diverges instead of converging to a minimum with large learning rate.



```
Epoch 1, Loss 187.386368
  Params: tensor([13.4659,  0.2100])
  Grad:  tensor([-1346.5909, -21.0000])
Epoch 2, Loss 497449.718750
  Params: tensor([-738.3428, -13.2925])
  Grad:  tensor([75180.8672, 1350.2495])
Epoch 3, Loss 1550619136.000000
  Params: tensor([41237.3320,  738.8163])
  Grad:  tensor([-4197567.5000, -75210.8828])
...
Epoch 10, Loss 4435733517345685329666721257095168.000000
  Params: tensor([-6.9746e+16, -1.2497e+15])
  Grad:  tensor([7.0995e+18, 1.2721e+17])
Epoch 11, Loss 13827561903253331560802272988415131648.000000
  Params: tensor([3.8941e+18, 6.9777e+16])
  Grad:  tensor([-3.9639e+20, -7.1026e+18])
...
tensor([-2.1742e+20, -3.8958e+18])
```

A Thermometer Example

- The effect of learning rate:
 - A smaller learning rate



The behavior is now stable.

```
Epoch 1, Loss 187.386368
  Params: tensor([0.1347, 0.0021])
  Grad:  tensor([-1346.5909, -21.0000])
Epoch 2, Loss 57.551537
  Params: tensor([0.1928, 0.0028])
  Grad:  tensor([-581.3164, -7.2875])
Epoch 3, Loss 33.357315
  Params: tensor([0.2179, 0.0030])
  Grad:  tensor([-250.9694, -1.3682])
...
Epoch 10, Loss 27.809898
  Params: tensor([0.2369, 0.0011])
  Grad:  tensor([-0.7568, 3.1148])
Epoch 11, Loss 27.808882
  Params: tensor([0.2370, 0.0008])
  Grad:  tensor([-0.3585, 3.1219])
...
Epoch 99, Loss 27.722918
  Params: tensor([ 0.2375, -0.0267])
  Grad:  tensor([-0.0560, 3.1225])
Epoch 100, Loss 27.721943
  Params: tensor([ 0.2375, -0.0270])
  Grad:  tensor([-0.0559, 3.1224])
tensor([ 0.2375, -0.0270])
```

A Thermometer Example

- Iterating to fit the model
 - We call a training iteration during which we update the parameters for all of our training samples an *epoch*.

```
def training_loop(n_epochs, learning_rate, params, t_u, t_c):  
    for epoch in range(1, n_epochs + 1):  
        w, b = params  
  
        t_p = model(t_u, w, b)          ◀— Forward pass  
        loss = loss_fn(t_p, t_c)  
        grad = grad_fn(t_u, t_c, t_p, w, b)  ◀— Backward pass  
  
        params = params - learning_rate * grad  
  
        print('Epoch %d, Loss %f' % (epoch, float(loss)))  
  
    return params
```

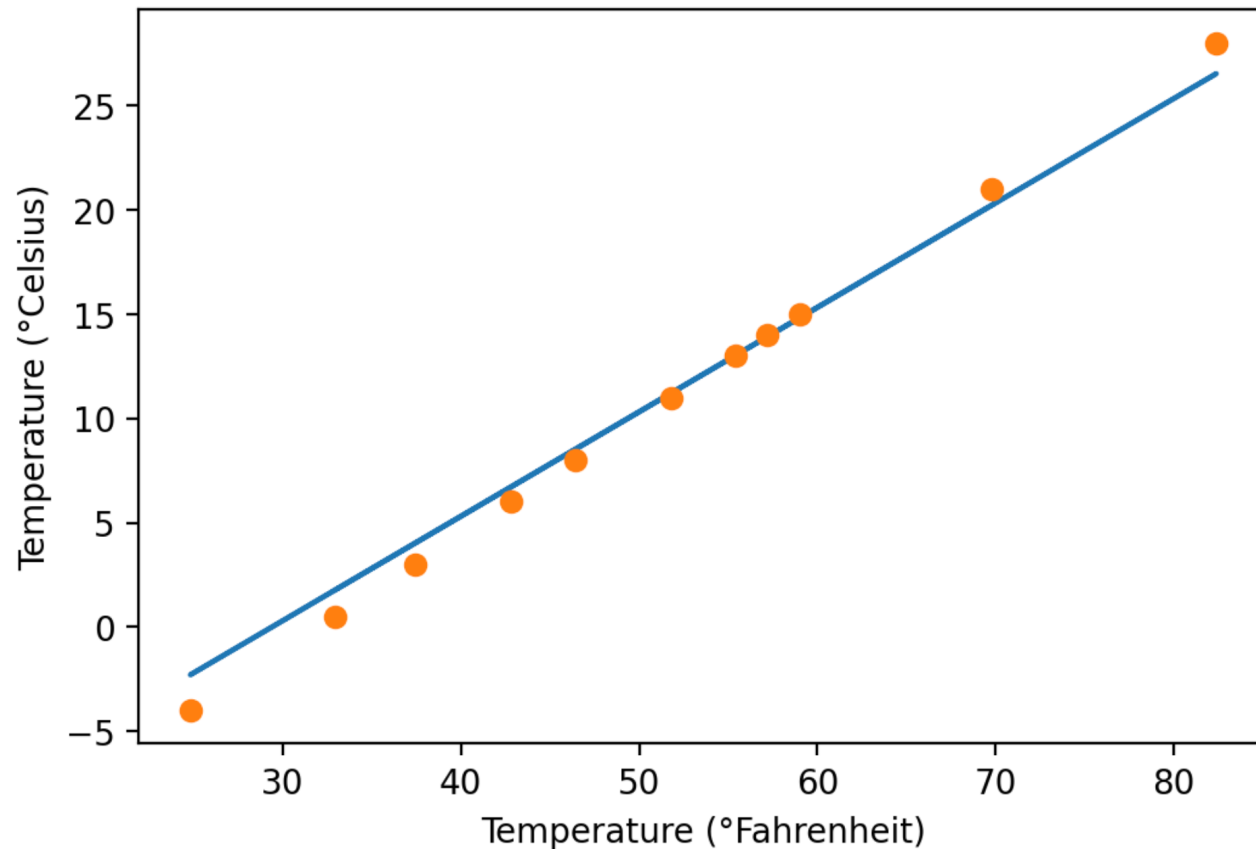
A Thermometer Example

- The effect of epochs:
 - Training the model 100000 epochs

```
Epoch 1, Loss 187.386368
Epoch 2, Loss 57.551537
Epoch 3, Loss 33.357315
...
Epoch 10, Loss 27.809898
Epoch 11, Loss 27.808882
...
Epoch 99, Loss 27.722918
Epoch 100, Loss 27.721943
...
Epoch 4000, Loss 24.167810
Epoch 5000, Loss 23.332361
Epoch 10000, Loss 19.568876
Epoch 100000, Loss 0.825063
tensor([ 0.5007, -14.7155])
```

A Thermometer Example

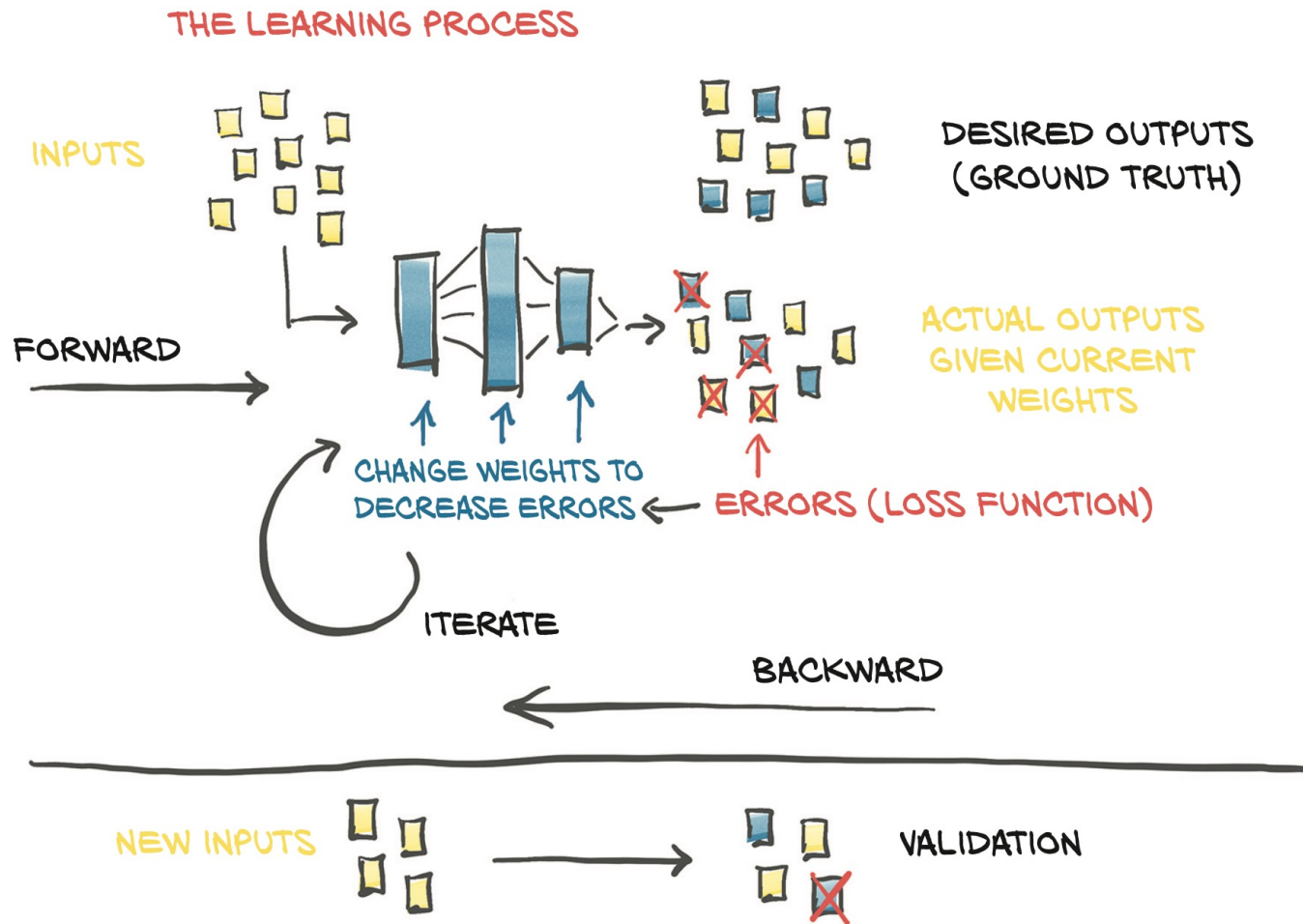
- Visualize the trained model



A Thermometer Example

We computed the gradient of a composition of functions—the model and the loss—with respect to their innermost parameters (w and b) by propagating derivatives backward using the *chain rule*.

A Complete Training Loop

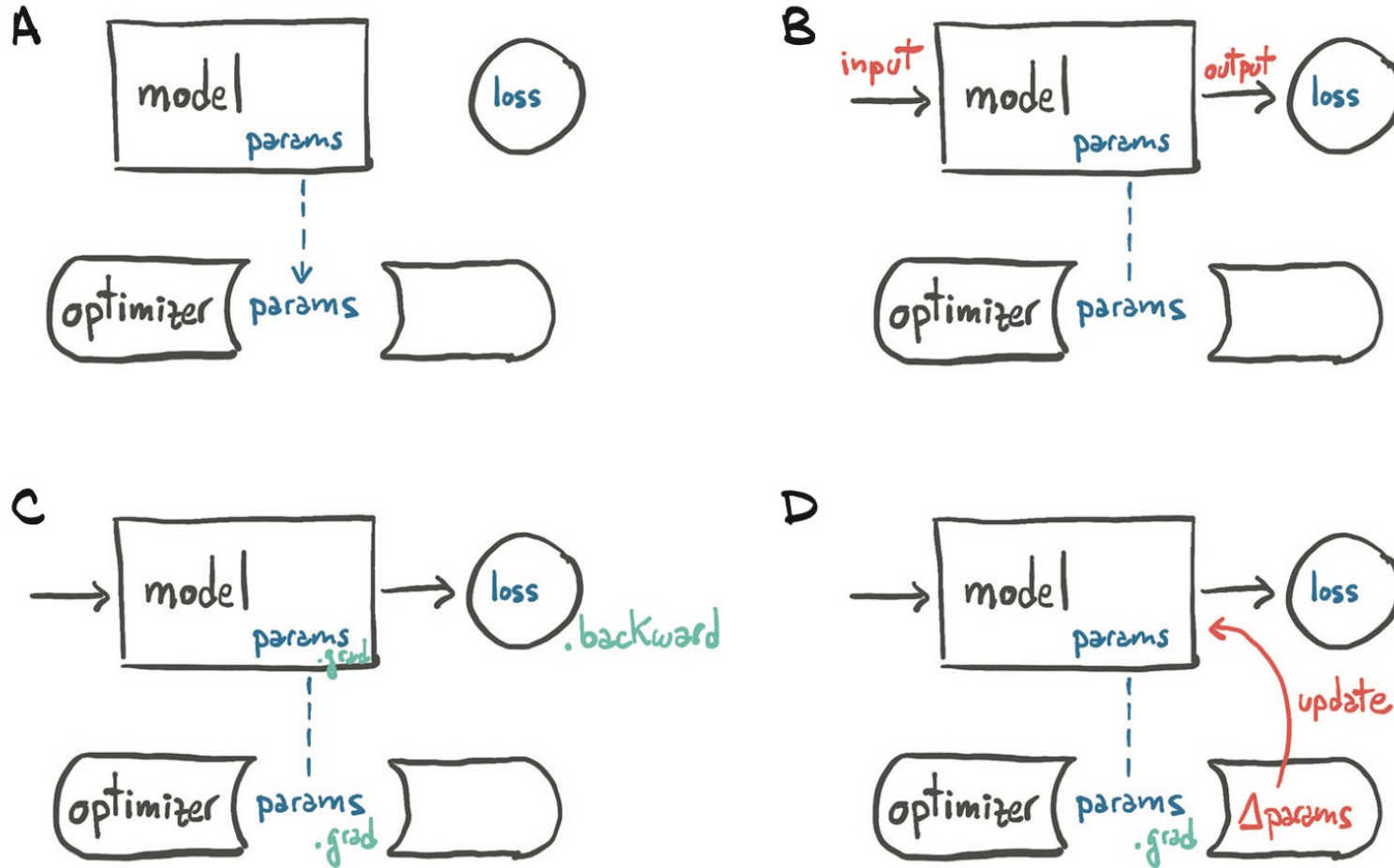


- Step 1: Define model
- Step 2: Define the loss function
- Forward pass
 - Step 3: Initialize the parameters
 - Step 4: Input data
- Backward pass
 - Step 5: Calculate the loss
 - Step 6: Calculate the gradient
 - Step 7: Update parameters

PyTorch Optimizer

- Optimizers in PyTorch: 'ASGD', 'Adadelata', 'Adagrad', 'Adam', 'AdamW', 'Adamax', 'LBFGS', 'Optimizer', 'RMSprop', 'Rprop', 'SGD', 'SparseAdam'.

PyTorch Optimizer



Forward pass

Backward pass

PyTorch Optimizer

■ Stochastic Gradient Descent (SGD) and Adam examples in Google Colab

```
# Define a SGD optimizer
params = torch.tensor([0.0, 0.0], requires_grad=True)
learning_rate = 1e-4
optimizer = optim.SGD([params], lr=learning_rate)

training_loop(n_epochs = 100000, optimizer = optimizer, params = params, t_u = t_u, t_c = t_c)
```

```
Epoch 5000, Loss 23.332361
Epoch 10000, Loss 19.568876
Epoch 15000, Loss 16.412432
Epoch 20000, Loss 13.765119
Epoch 25000, Loss 11.544819
Epoch 30000, Loss 9.682666
Epoch 35000, Loss 8.120876
Epoch 40000, Loss 6.810985
Epoch 45000, Loss 5.712368
Epoch 50000, Loss 4.791006
Epoch 55000, Loss 4.018200
Epoch 60000, Loss 3.370096
Epoch 65000, Loss 2.826465
Epoch 70000, Loss 2.370594
Epoch 75000, Loss 1.988215
Epoch 80000, Loss 1.667524
Epoch 85000, Loss 1.398525
Epoch 90000, Loss 1.172966
Epoch 95000, Loss 0.983774
Epoch 100000, Loss 0.825063
tensor([ 0.5007, -14.7155], requires_grad=True)
```

```
# Using a Adam optimizer
params = torch.tensor([0.0, 0.0], requires_grad=True)
learning_rate = 1e-3
optimizer = optim.Adam([params], lr=learning_rate)

training_loop(n_epochs = 30000, optimizer = optimizer, params = params, t_u = t_u, t_c = t_c)
```

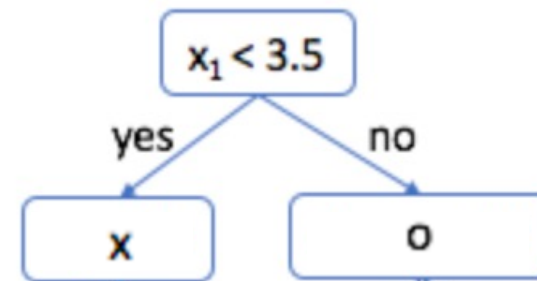
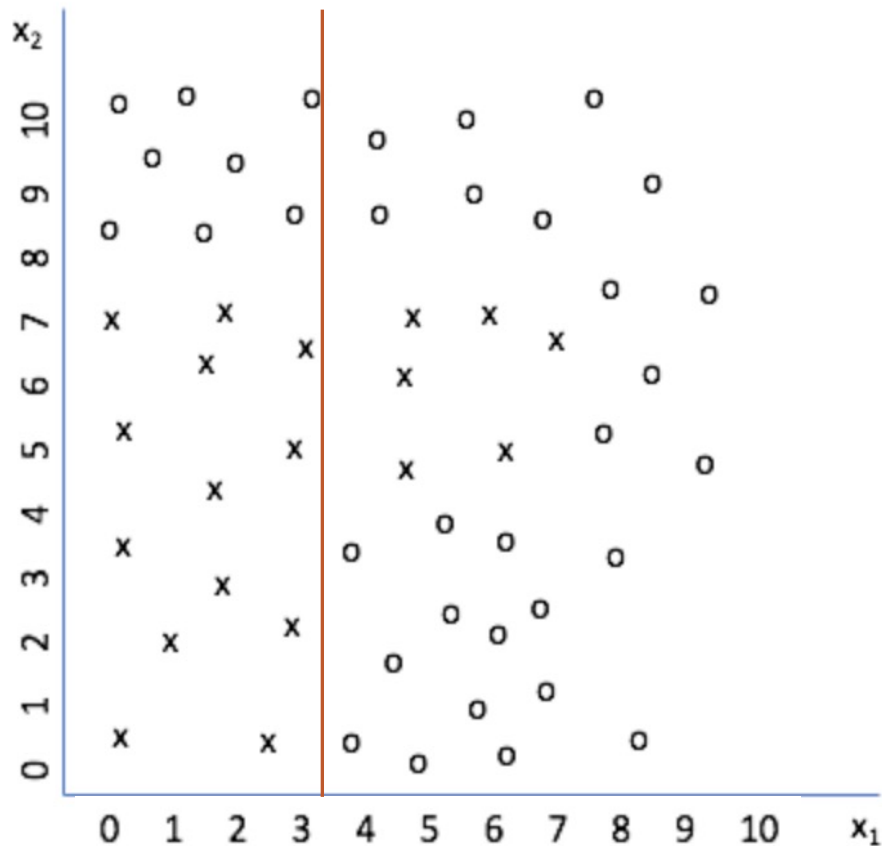
```
Epoch 5000, Loss 17.977310
Epoch 10000, Loss 8.228080
Epoch 15000, Loss 2.417738
Epoch 20000, Loss 0.171821
Epoch 25000, Loss 0.000000
Epoch 30000, Loss 0.000000
tensor([ 0.5556, -17.7778], requires_grad=True)
```

Generalization and Overfitting

- Generalization is the property of a model or modeling process, whereby the model applies to data that were not used to build the model.
- Overfitting is the tendency of data mining procedures to tailor models to the training data, at the expense of generalization to previously unseen data points.

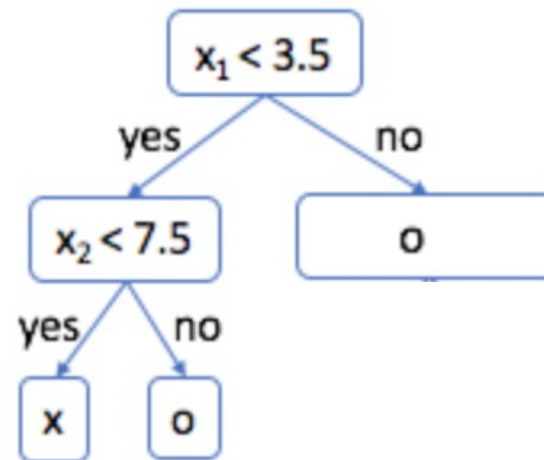
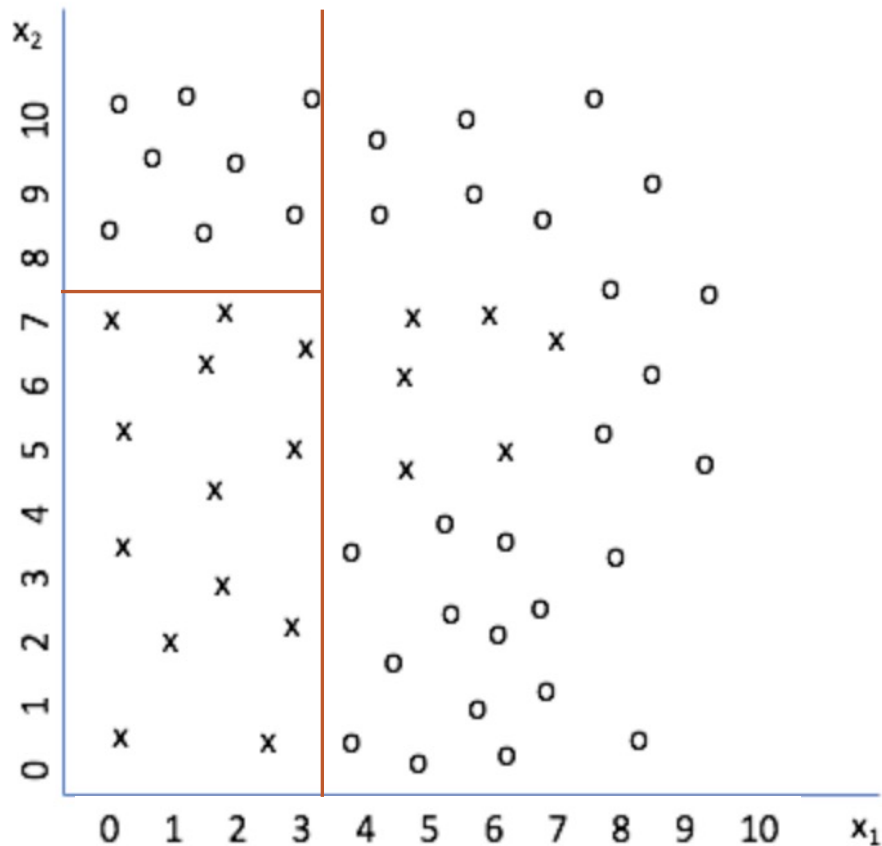
Generalization and Overfitting

■ Decision Tree



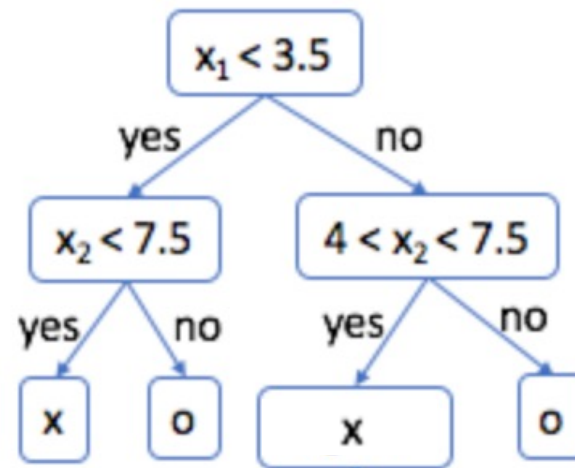
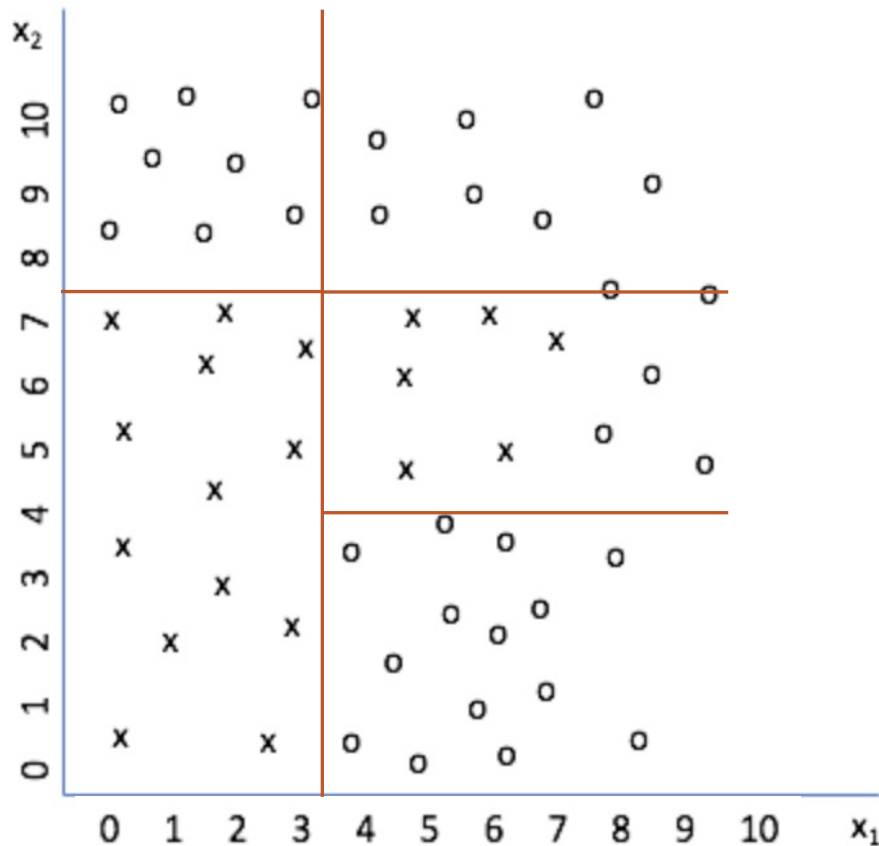
Generalization and Overfitting

■ Decision Tree



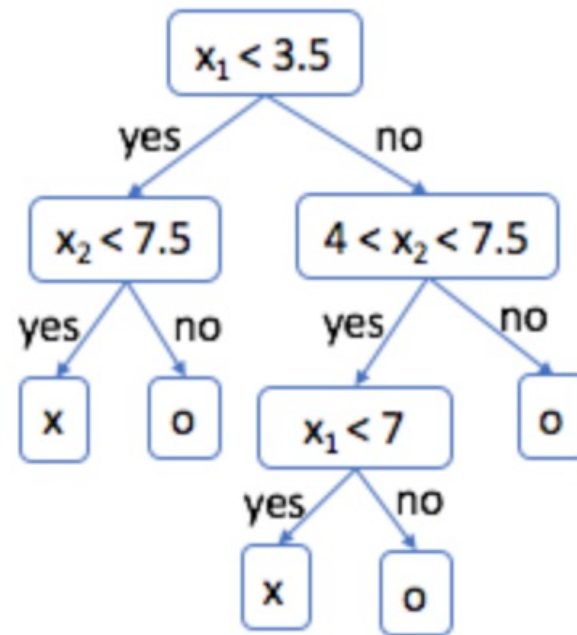
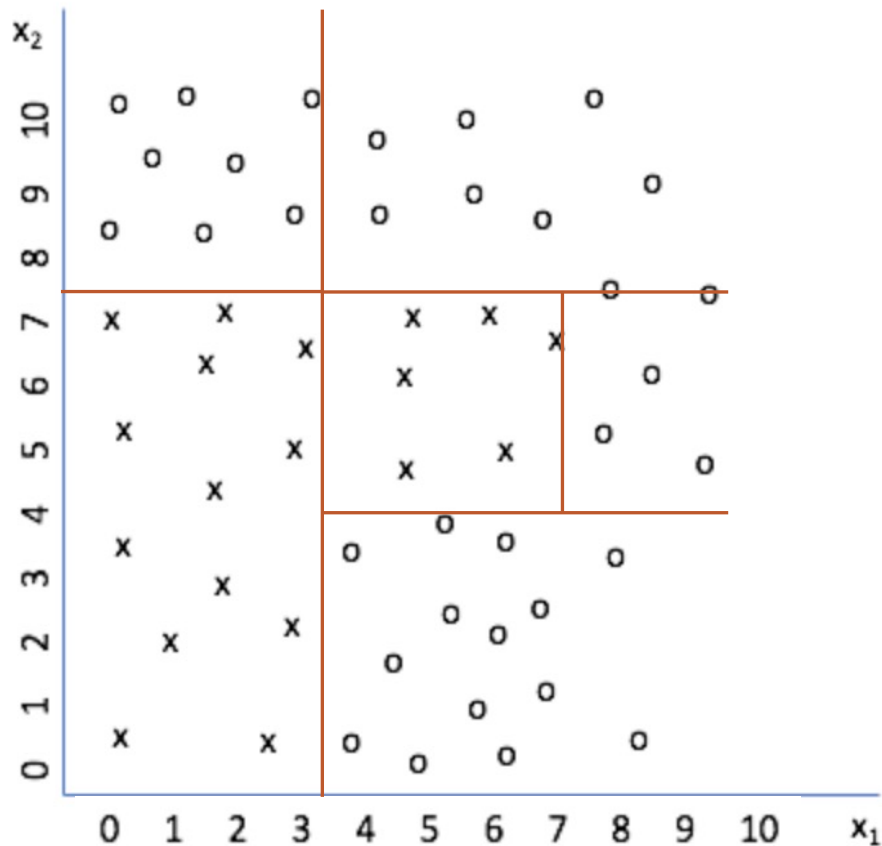
Generalization and Overfitting

■ Decision Tree



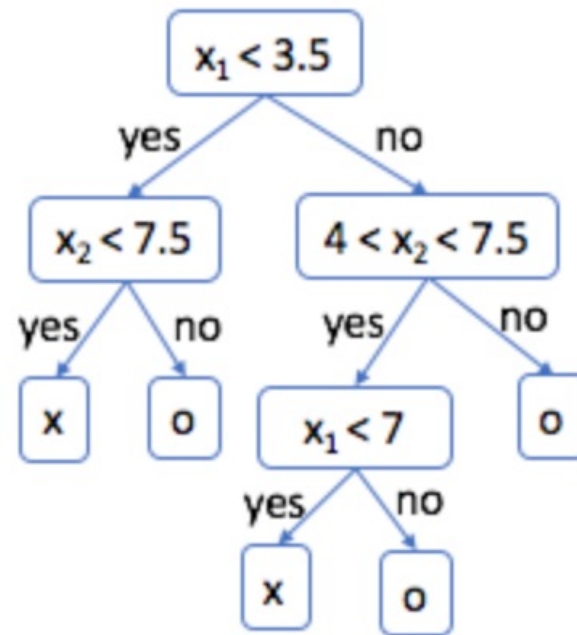
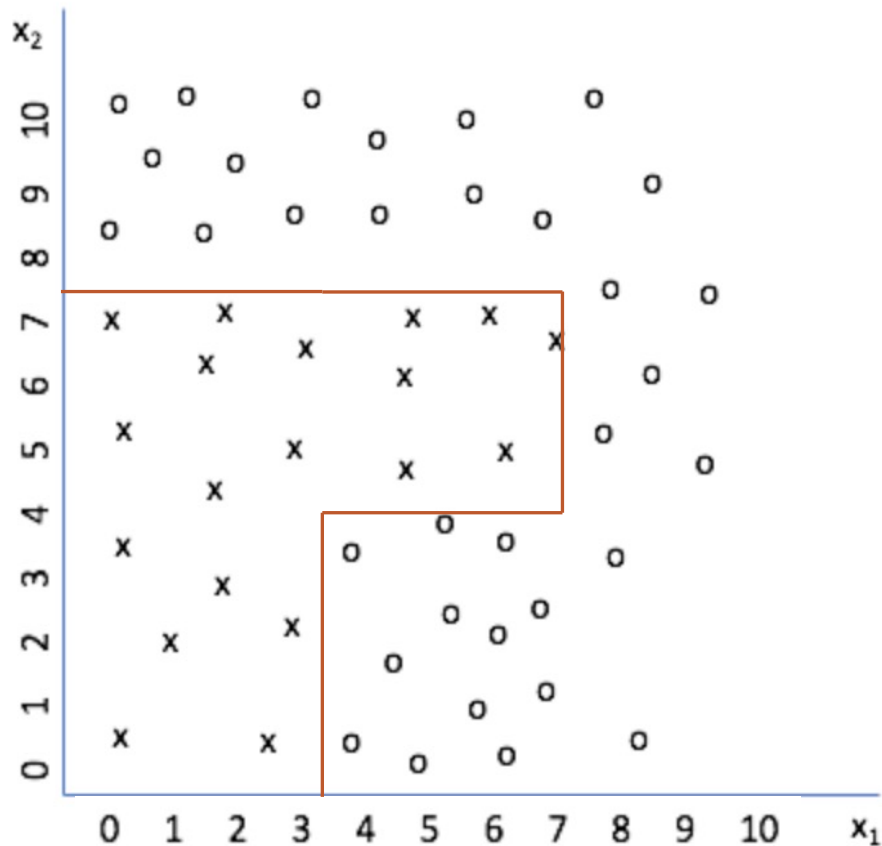
Generalization and Overfitting

■ Decision Tree

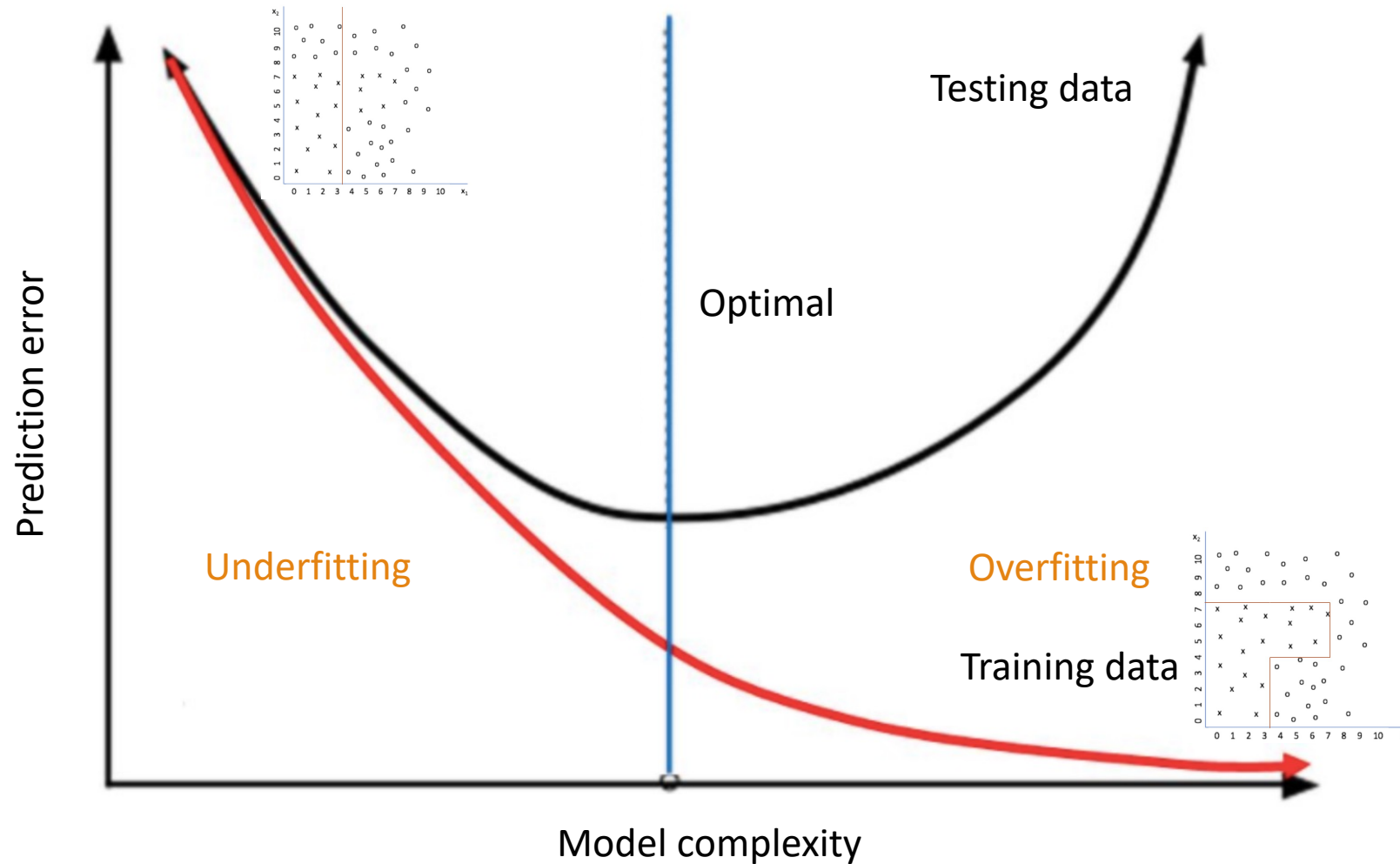


Generalization and Overfitting

■ Decision Tree

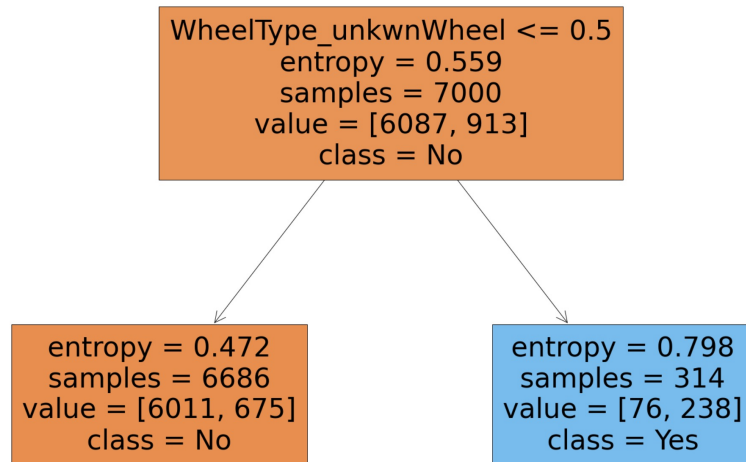


Generalization and Overfitting



Generalization and Overfitting

- Comparing the model **performances** on **training** and **testing** data
 - Max depth = 1



Training performance

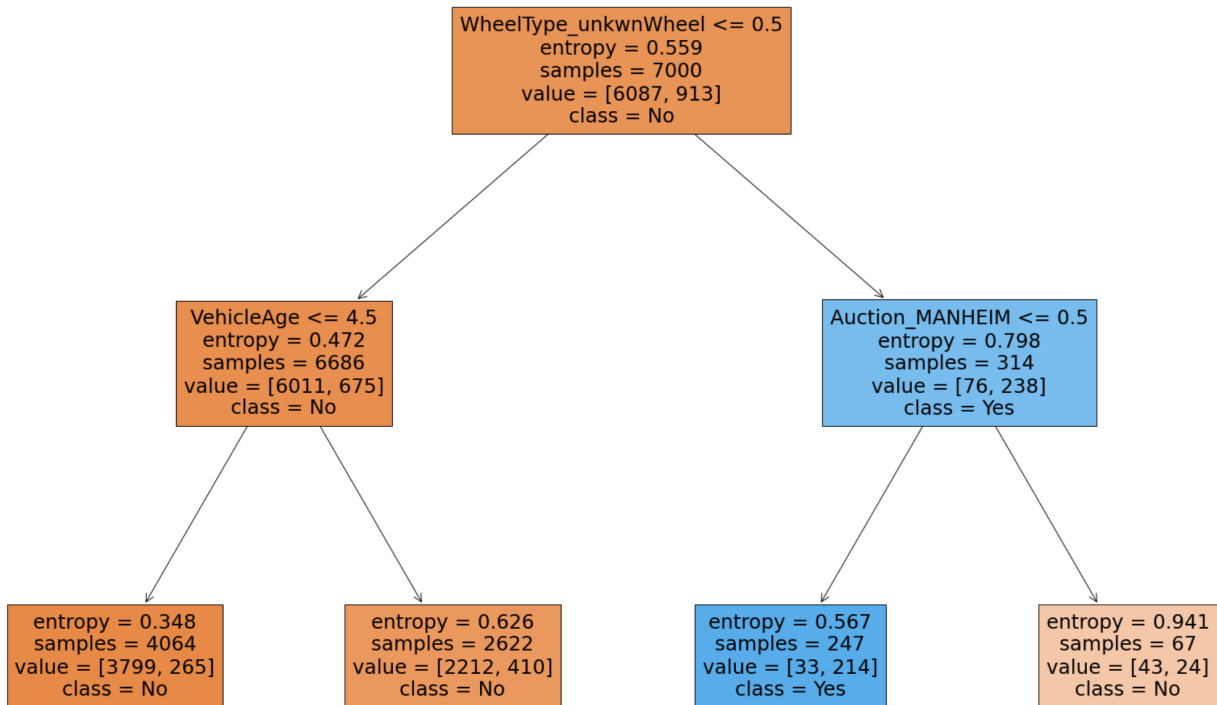
	precision	recall	f1-score	support
No	0.90	0.99	0.94	6087
Yes	0.76	0.26	0.39	913
accuracy			0.89	7000
macro avg	0.83	0.62	0.66	7000
weighted avg	0.88	0.89	0.87	7000

Testing performance

	precision	recall	f1-score	support
No	0.90	0.99	0.94	2618
Yes	0.76	0.26	0.39	382
accuracy			0.90	3000
macro avg	0.83	0.62	0.67	3000
weighted avg	0.88	0.90	0.87	3000

Generalization and Overfitting

- Comparing the model **performances** on **training** and **testing** data
 - Max depth = 2



Training performance

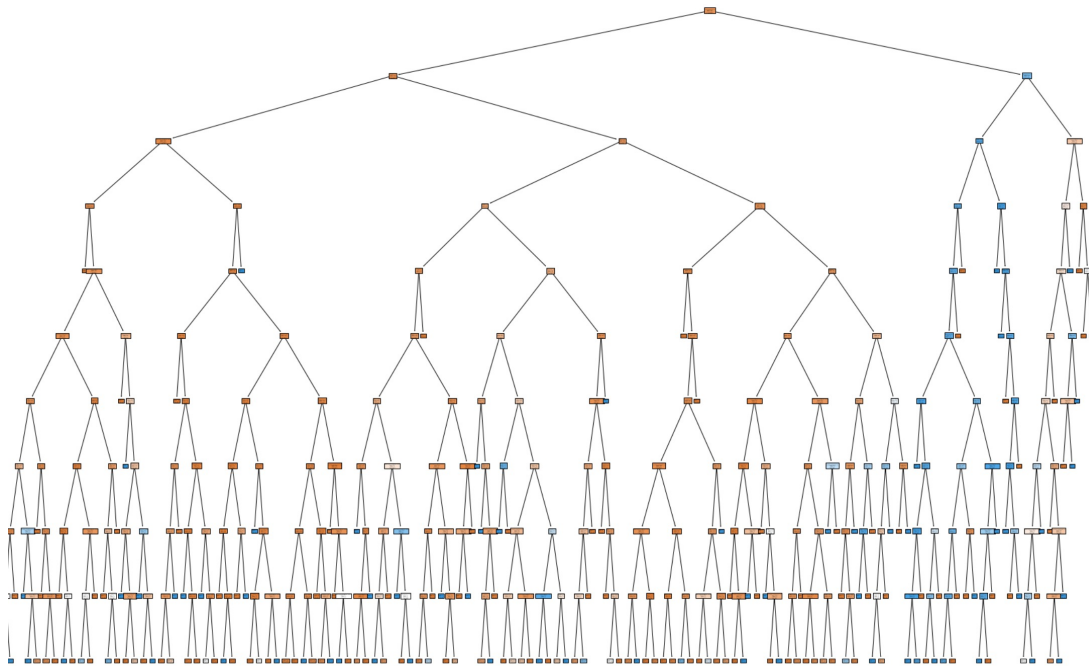
	precision	recall	f1-score	support
No	0.90	0.99	0.94	6087
Yes	0.87	0.23	0.37	913
accuracy			0.90	7000
macro avg	0.88	0.61	0.66	7000
weighted avg	0.89	0.90	0.87	7000

Testing performance

	precision	recall	f1-score	support
No	0.90	1.00	0.94	2618
Yes	0.90	0.22	0.36	382
accuracy			0.90	3000
macro avg	0.90	0.61	0.65	3000
weighted avg	0.90	0.90	0.87	3000

Generalization and Overfitting

- Comparing the model **performances** on **training** and **testing** data
 - Max depth = 10



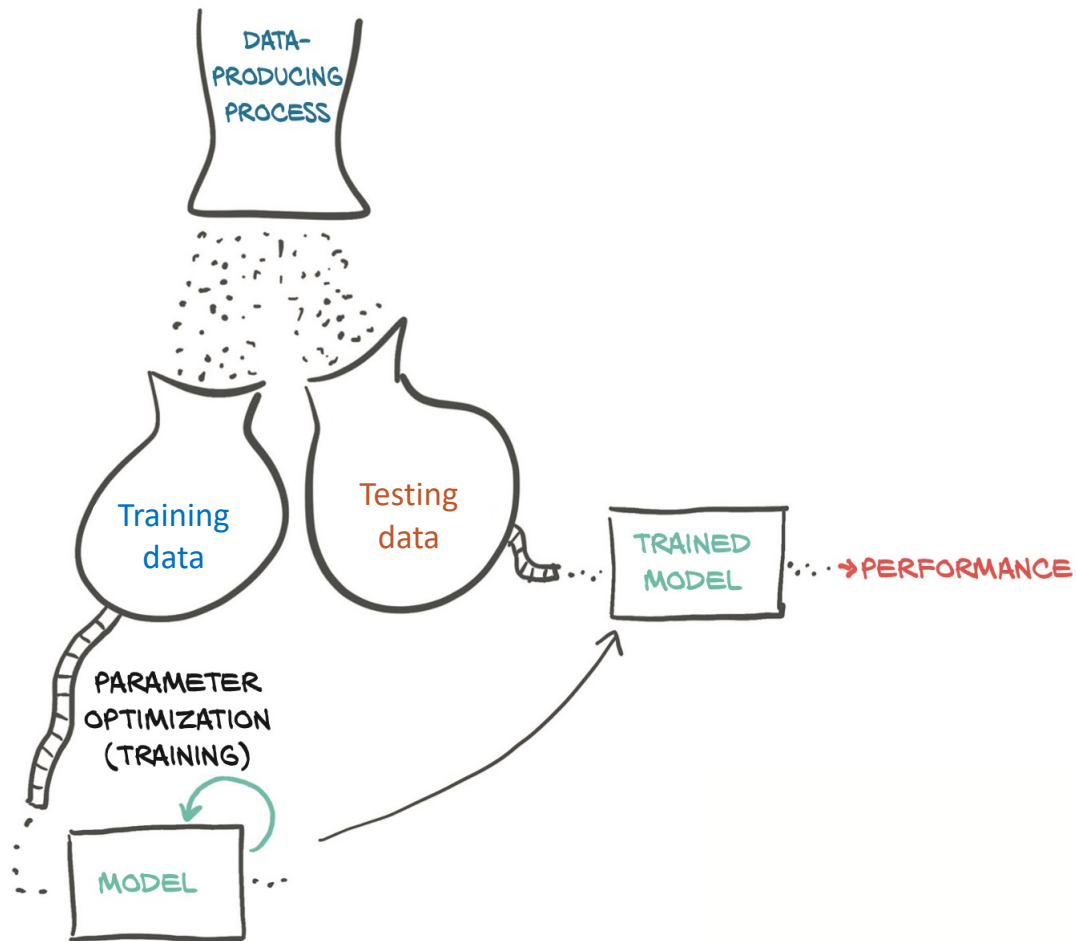
Training performance

	precision	recall	f1-score	support
No	0.92	1.00	0.96	6087
Yes	0.94	0.41	0.57	913
accuracy			0.92	7000
macro avg	0.93	0.70	0.76	7000
weighted avg	0.92	0.92	0.90	7000

Testing performance

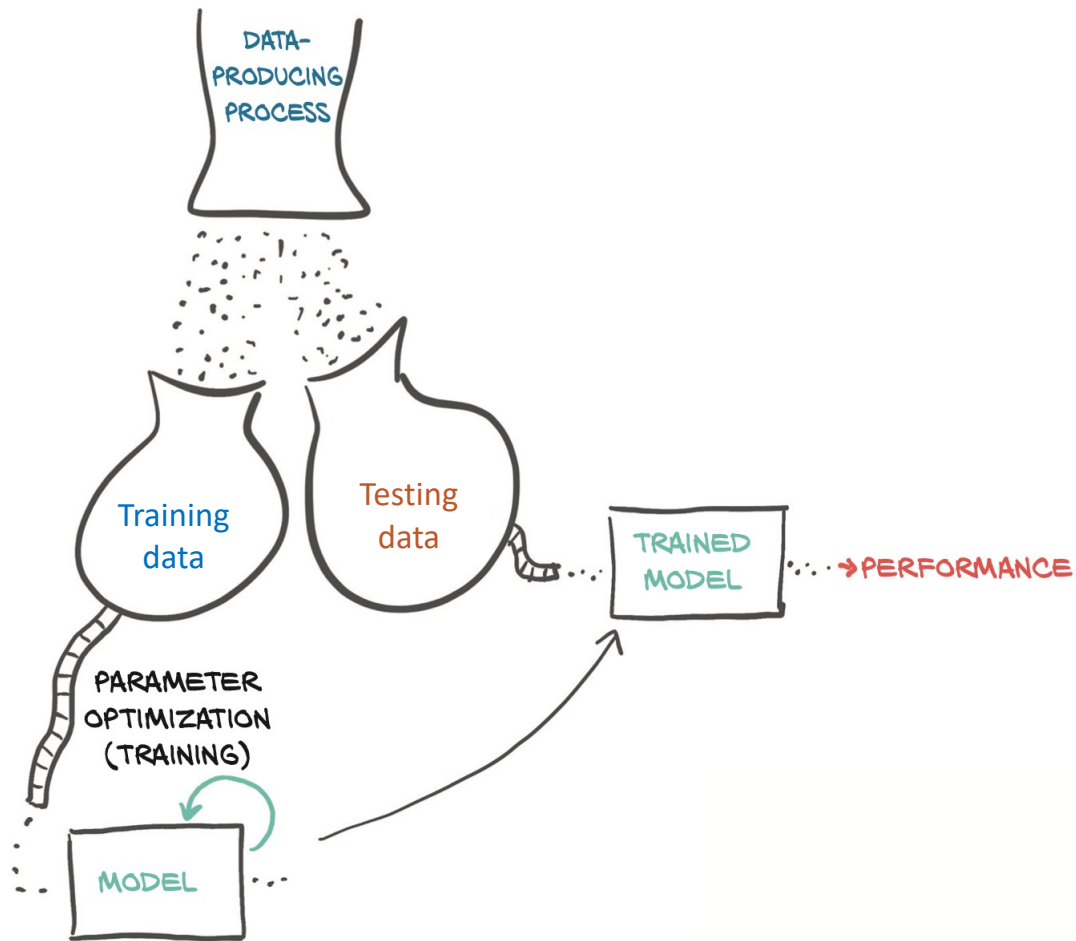
	precision	recall	f1-score	support
No	0.90	0.97	0.93	2618
Yes	0.56	0.25	0.35	382
accuracy			0.88	3000
macro avg	0.73	0.61	0.64	3000
weighted avg	0.86	0.88	0.86	3000

Generalization and Overfitting



- Evaluate the **training** loss
 - The training loss will tell us if our model can fit the training set at all—in other words, if our model has enough capacity to process the relevant information in the data.
 - Training loss would stop decreasing (or decrease slowly) before approaching zero
 - If the training loss is not decreasing, chances are the model is too simple for the data
 - The other possibility is that our data just doesn't contain meaningful information that lets it explain the output

Generalization and Overfitting



- Generalize to the **testing** set
- If the loss evaluated in the testing set doesn't decrease along with the training set, it means our model is not *generalizing* to samples outside this precise set
 - if the training loss and the testing loss diverge, we're overfitting

Generalization and Overfitting

- Additionally evaluate the testing loss at every epoch

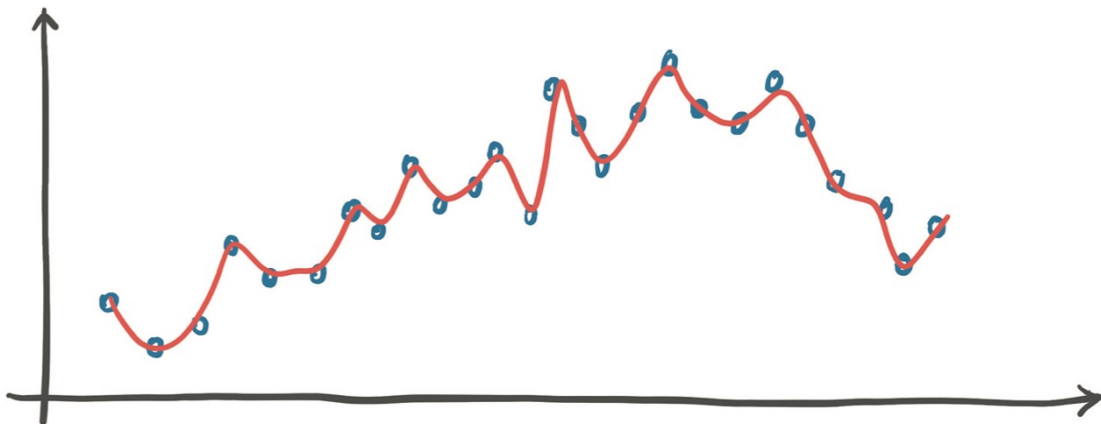
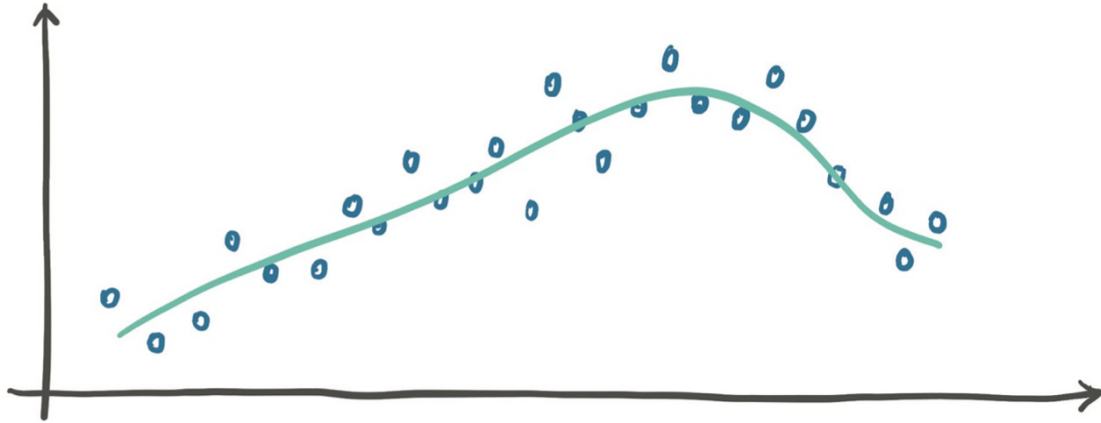
```
def training_loop(n_epochs, optimizer, params, train_t_u, test_t_u, train_t_c, test_t_c):  
    for epoch in range(1, n_epochs + 1):  
        train_t_p = model(train_t_u, *params)  
        train_loss = loss_fn(train_t_p, train_t_c)  
  
        test_t_p = model(test_t_u, *params)  
        test_loss = loss_fn(test_t_p, test_t_c)  
  
        optimizer.zero_grad()  
        train_loss.backward()  
        optimizer.step()  
  
        if epoch <= 3 or epoch % 5000 == 0:  
            print(f"Epoch {epoch}, Training loss {train_loss.item():.4f}, " f" Testing loss {test_loss.item():.4f}")  
  
    return params
```

Generalization and Overfitting

- We expect a model to perform better on the training set, since the model parameters are being shaped by the training set.

```
Epoch 1, Training loss 226.2500, Testing loss 12.5000
Epoch 2, Training loss 224.5845, Testing loss 12.4891
Epoch 3, Training loss 222.9259, Testing loss 12.4803
Epoch 5000, Training loss 13.3756, Testing loss 53.1700
Epoch 10000, Training loss 6.4599, Testing loss 25.6979
Epoch 15000, Training loss 2.1257, Testing loss 8.4567
Epoch 20000, Training loss 0.2359, Testing loss 0.9383
Epoch 25000, Training loss 0.0000, Testing loss 0.0000
Epoch 30000, Training loss 0.0000, Testing loss 0.0000
tensor([ 0.5556, -17.7778], requires_grad=True)
```


Generalization and Overfitting



- We could have decided to fit the data with a more complicated function, like a really large neural network.
- It could generate a model meandering its way through the data points, just because it pushes the loss very close to zero (Higher chance of overfitting).

Generalization and Overfitting

Causes for Overfitting:

- Training data is not a good representation of testing (new) data
 - Insufficient training data
 - Noises in data: inconsistent class labels for the same values in feature set (input attributes)
 - Outliers in data: the number of samples with a given combination of class labels and feature values is small.
- An algorithm's inability to avoid overfitting noises/outliers or to train generalizable models via small amounts of training data
 - Complex model

Generalization and Overfitting

Avoidance of Overfitting

- Data strategies
 - Secure sufficient data
 - Identify and handle potential outliers and noises
- Evaluation strategies
 - **Identifying overfitting**: comparing the model **performance** on **training** and **testing** data
 - **Avoid big gap between the training and testing accuracy**
- Model strategies
 - Select proper algorithm and manage model complexity
 - Compare different algorithms
 - Lower model complexity via method-specific parameters

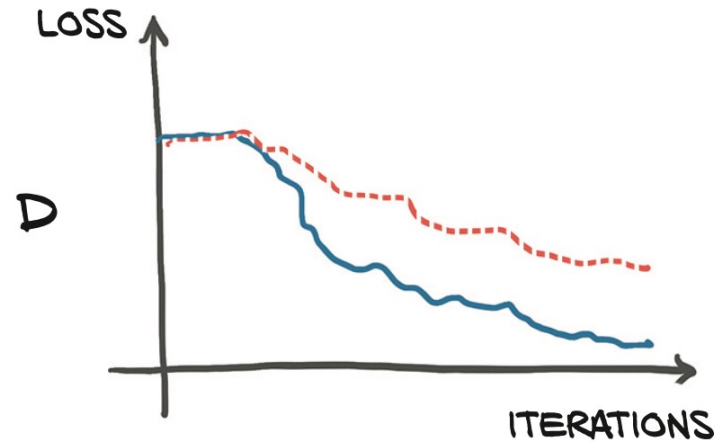
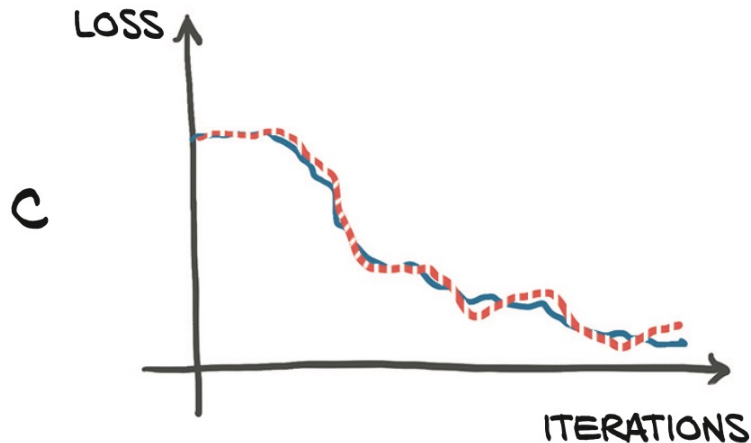
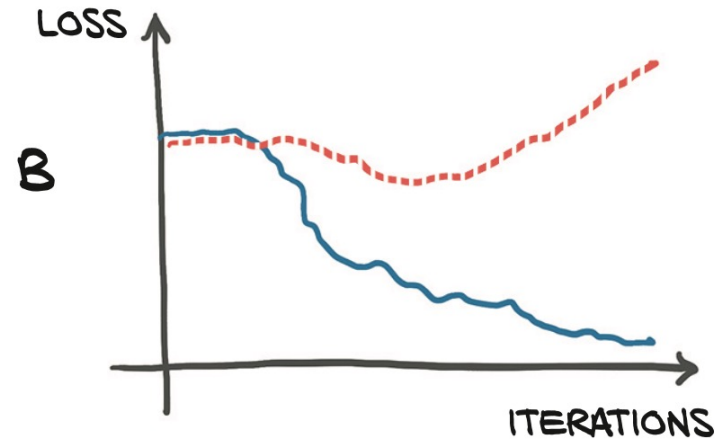
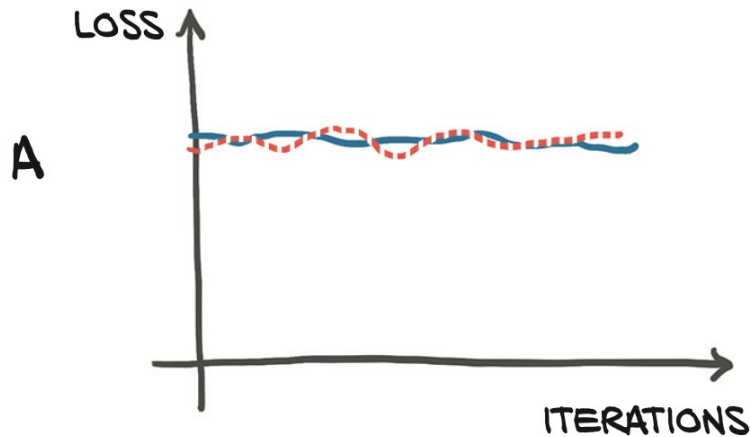
Generalization and Overfitting

- We should make sure the model generalizes well on the new data points.
 - Get large enough data
 - Adding penalization terms to the loss function
 - Adding dropout

Generalization and Overfitting

- Our main goal is to see both the training loss and the testing loss decreasing.
- While ideally both losses would be roughly the same value, as long as the testing loss stays reasonably close to the training loss, we know that our model is continuing to learn generalized things about our data.

Generalization and Overfitting



- A. Underfitting
- B. Overfitting
- C. Ideal
- D. Acceptable